FPGA Acceleration of Multilevel ORB Feature Extraction for Computer Vision

Josh Weberruss*‡, Lindsay Kleeman*, David Boland† and Tom Drummond*‡
*Monash University
†University of Sydney
‡Australian Centre of Excellence for Robotic Vision

***Abstract*—In this paper, we present the first multilevel implementation of the Harris-Stephens corner detector and the ORB feature extractor running on FPGA hardware, for computer vision and robotics applications. ORB is a fundamental component of many robotics applications, and requires significant computation. The design has been validated both in behavioural simulation and in implementation on an Arria V FPGA connected to a desktop PC via PCI-Express. A Linux kernel-mode driver and userspace library allow integration of the acceleration hardware into C++ programs. The device has significantly higher throughput than a CPU implementation (150 MPixel/s vs 27 MPixel/s) and a GPU implementation (40 MPixel/s), with much lower power draw (5.3 W vs 145 W). This throughput is equivalent to 72 fps at 1920×1080 or 488 fps at 640×480.**

I. Introduction

A basic problem in robotics is the Simultaneous Localisation and Mapping (SLAM) problem—given sensor input, the robot must determine where it is in the world and map its environment. Using visual input to drive SLAM is a promising area of research at present, and several SLAM algorithms are designed expressly to work with visual input, eg ORB-SLAM [1].

ORB-SLAM works by finding correspondences between corners in video frames across time; that is, it finds corners in images, then attempts to find similar-looking corners in later frames of video. Corners are detected using the FAST [2] and Harris-Stephens [3] corner detectors, and matched using the ORB (Oriented FAST and Rotated BRIEF) feature extractor algorithm [4].

The detection of corners and their matching is a significant computational load for the ORB-SLAM algorithm. This processing is done on a CPU core in [1], or offloaded to a GPU in later implementations.

Current CPU and GPU implementations, and even state-of-the-art FPGA implementations, are unable to run at realtime 60 fps for 1920×1080 images [5], [6] (see table III), which means that some input frames must be dropped, at the cost of temporal resolution. This is a particular problem with rapid camera motion, as skipping a frame can mean that the processed images are dramatically different, complicating localisation.

We propose in this work an FPGA-based hardware accelerator device for the purpose of offloading Harris-Stephens corner detection and ORB feature extraction from the CPU and GPU, running faster and using significantly less electrical energy per pixel than either alternative, while being able to handle images of up to 65535×65535 pixels.

The device, which we call a ***Tarsier***, after the family of primates with excellent vision, has been implemented on an Altera Arria V GX FPGA Starter Kit. This board connects to a desktop PC via a PCI-Express 2.0 x4 link and contains an FPGA with 360,000 LEs.

Corresponding Author - josh.weberruss@monash.edu

A. Source Code

The Verilog HDL code, the Linux driver, and accompanying utilities are available under a Mozilla Public Licence at https://github.com/raiker/tarsier.

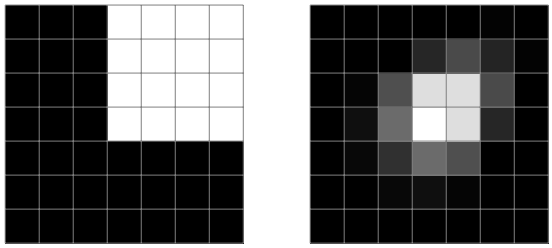
B. Previous Work

Several groups have previously investigated hardware acceleration of sparse feature detection and matching. In 2010 (the year before ORB was published), Bouris et al implemented the SURF descriptor in a system that processed 640×480 image frames at 56 fps [7], equivalent to 17 MPixel/s. Lee [5] demonstrated an implementation of FAST corner detection and ORB feature extraction in 2013 that performed at 108 fps for 640×480 images. This system used only a single scale level and executes at 3 cycles/pixel. The feature detector and extractor operate by wide random access to a memory containing the image data.

In 2015, we published an implementation [6] of ORB using the Harris-Stephens corner detector that managed 1 cycle/pixel, again at a single scale level. This implementation made use of a sliding window structure, or 2D shift register, in order to minimise memory access latency.

[8] investigates the relative performance of implementations of the BRIEF and BRISK algorithms across CPU, GPU and FPGA platforms. Despite mentioning ORB, the authors do not investigate its performance. Their implementations of FAST, BRIEF and BRISK use the sliding window structure to increase throughput, but only single-level implementations are considered.

While a single-level implementation of ORB does successfully match some keypoints, the initial publication in [4] makes clear that multilevel image pyramids are necessary to exhibit maximum performance. To that end, we take inspiration from the implementation of Harris-Stephens and ORB in [6] to



(a) Input image

(b) Corresponding Harris-Stephens corner scores (normalised)

Fig. 1: Corner scores emitted by the Harris-Stephens detector. White indicates higher values. The corner scores on the right correspond to the pixels on the left.

provide a multilevel ORB system. A multilevel system is able to detect and extract features at different scales, enabling better handling of camera motion (see section I-E).

C. Contributions of this Work

We improve on the implementation of corner detection and matching in [6] by adding multiscale feature detection and extraction using an image pyramid. We investigate the parameters of such designs, in particular the number and spacing of the levels in the pyramid (see section I-E), and report on the performance of the Tarsier device.

D. Computer Vision Algorithms

We make use of two different computer vision algorithms in our design, and make reference to a third, which is used in the OpenCV implementations. Figure 2 shows the sequence of the three.

1) Harris-Stephens: The Harris-Stephens corner detector [3] examines the local environment of a pixel and returns a score, indicating how confident it is that the pixel represents a corner. "Corner" here is defined as a point where two edges cross, as determined by local pixel intensity gradient. The Harris-Stephens detector is computationally intensive for a CPU, so in most applications, another algorithm (typically FAST (see section I-D2)) is used to supply points that could possibly be corners for further investigation.

Within corners, there is typically a region of pixels with high corner scores. In most applications, we only want one corner to be returned, as opposed to a cluster of adjacent points, so an additional pass is made after corner scores are calculated to locate the local maximum. The algorithm used, non-maximum suppression, excludes all the corner candidates that are adjacent to a corner candidate with a higher score, which is more likely to be the corner.

Figure 1 shows the calculated corner scores for a simple corner. There is a region of high corner scores in the centre of the image, with the peak located in the centre of the window. Non-maximum suppression would select this centre point as a corner, and pass it on to the next stage.

Since each corner is assigned a score, it is possible to select only very strong corners by setting a threshold corner score, below which the corner is ignored.

2) FAST: FAST (Features by Accelerated Segment Test) [2] is an algorithm for quickly filtering out points which are unlikely to be a corner, saving work for Harris-Stephens. FAST examines a small ring of 16 pixels around a potential centre pixel and includes only points where there is a run of at least 9 pixels that are all brighter or darker than the centre point. This admits only points that are on an edge or a corner. Harris-Stephens then filters out points that are just on edges, leaving only corners.

Since the hardware implementation of Harris-Stephens runs on every single pixel in the input image with no additional logic costs, it is unnecessary to implement FAST, as Harris-Stephens does the same job more robustly. It is possible the FAST will filter out a corner that Harris-Stephens will allow, but we consider that to be an advantage, as Harris-Stephens is a more robust metric.

3) ORB: ORB (Oriented FAST and Rotated BRIEF) [4] is a feature extraction algorithm. Given a corner as input, ORB calculates a "hash" of the environment, allowing the corner to be compared to other corners in other images.

ORB produces as its output a 256-bit binary string, termed a descriptor. Each bit in the descriptor represents a comparison of the intensity values of two pixels at specific coordinates around the corner—if pixel A is brighter than pixel B, the bit will be set, otherwise it will be reset. If two photographs are taken of the one object, from different angles, it is likely that their environments will look similar, and thus most of the comparisons will give the same output bit, and the descriptors will be similar.

Algorithm 1 ORB Feature Extraction

function ORB($image, x, y, angle$)
  for all i : RANGE(256) **do**
    $(x_a, y_a) \leftarrow samples_a[i]$
    $(x_b, y_b) \leftarrow samples_b[i]$
    $(rx_a, ry_a) \leftarrow$ VECTORROTATE$(x_a, y_a, angle)$
    $(rx_b, ry_b) \leftarrow$ VECTORROTATE$(x_b, y_b, angle)$
    $pix_a \leftarrow image[x + rx_a, y + ry_a]$
    $pix_b \leftarrow image[x + rx_b, y + ry_b]$
    $descriptor[i] \leftarrow pix_a > pix_b$
  end for
  return $descriptor$
end function

ORB has the property of rotation invariance, meaning that if a feature is rotated without changing scale, ORB will generate exactly the same descriptor. This is achieved by rotating the environment around the feature into a "canonical" direction before sampling. The image is rotated so that the centroid of the feature (in broad terms, the brightest point near the feature) is pointing upwards, and then the fixed pixel coordinates are sampled.

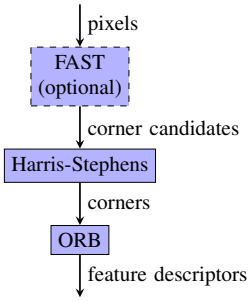


Fig. 2: Sequence of the FAST, Harris-Stephens and ORB operations. FAST is used in the OpenCV CPU and GPU implementations, but is not necessary in the Tarsier device.

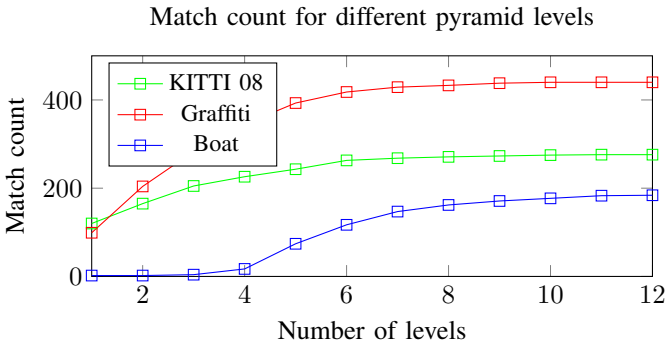


Fig. 3: Match counts with increasing pyramid heights for three different pairs of images (three levels per octave)

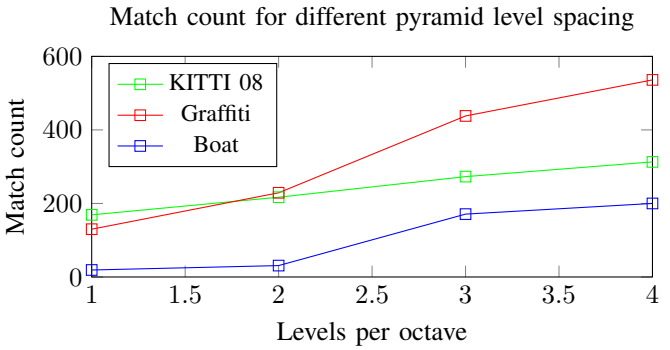


Fig. 4: Match counts with increasing pyramid levels per octave for three different pairs of images (9 levels total)

To compare descriptors between two corners, the Hamming distance measure is used (i.e. count of bits that differ). Descriptors that represent the same corner from two different viewpoints should have a low Hamming distance, and descriptors representing two different corners should have a high Hamming distance. We can apply a threshold to the Hamming distances to determine whether a given match is valid or not.

For hardware implementation details, please see section II-C.

4) Image Correspondences: To determine image correspondences between two frames of video, we first find all the corners in the images, as described in section I-D2, then pass these to ORB and generate their descriptors, as shown in figure 2. The output of this process is two lists of tuples of x-coordinate, y-coordinate and 256-bit descriptor.

We then match the features in the first frame to the features in the second using a brute-force algorithm, where for each feature in the first image, we find the feature in the second with the smallest Hamming distance, subject to a maximum threshold.

This algorithm has complexity $\mathcal{O}(n \cdot m)$, where n is the number of features in image 1 and m is the number of features in image 2.

This process is currently implemented in software, but will be put onto the FPGA in the near future.

E. Image Pyramid

Image pyramids were introduced in [9], and offer an effective method of processing images at multiple resolutions. As well as the original images, multiple additional downsampled (i.e. lower resolution) copies, at successively lower resolutions, are stored alongside. For certain algorithms, including Harris-Stephens corner detection and ORB feature extraction, which operate at fixed scale, running them on all the different levels of the pyramid can yield different outputs, often providing more information.

In the case of the Harris-Stephens detector, it is able to detect rounded corners with a certain maximum corner radius (given in pixels, dependent on the contrast and other factors). This might mean that the detector will miss a corner at the base scale of the image, but moving a few levels up the pyramid (lowering the resolution of the image) shrinks the corner radius and makes the corner detector able to find it.

ORB, too, operates on a fixed scale, with a window approximately 37 pixels in radius. This fixed scale is particularly significant in cases where the camera is moving towards or away from an object. Two successive frames of motion might show the object at two different sizes, thanks to perspective foreshortening. A single-scale implementation of ORB might generate two descriptors which are too different to match successfully in such a case. On the other hand, if ORB is run at all the different scale levels, there is likely to be a scale level where the pixel sizes of the features are very similar.

Figure 3 shows the effect on the number of matches generated by a simulated version of the Tarsier system with increasing image pyramid heights. The three pairs of images demonstrate different effects.

The series labelled “KITTI 08” matches two stereo frames from the start of sequence 8 of the KITTI dataset [10]. The difference between the viewpoints is only a small translation, with all features in one image being the same scale as the other. As can be seen, with increasing pyramid levels, the system is able to detect and match more features. This is entirely due to rounded corners becoming usable as they are downsampled and become sharper. From around 7 levels onwards, there are minimal gains, as almost all the features are matched.

The series labelled "Graffiti" is from Oxford Visual Geometry Group's "Affine Covariant Regions" Dataset [11]. The two images are slightly different scales and substantially rotated, with lots of high-contrast features at varying scales. With increasing numbers of levels in the pyramid, the number of matched features increases dramatically, until it flattens out at around 9 levels.

The final series, labelled "Boat", is also from the Oxford VGG dataset [11]. These images are much noisier and show dramatic differences in scale (the second frame has approximately $2\times$ the field of view of the first), as well as a slight rotation. Because of the significant difference in scale, the matcher struggles to find any matches at all with a short image pyramid—the ORB descriptor is sensitive to scale differences, and even downsampled, the features in the big image are still larger than those in the small image. Around 5 levels in, this starts to change, as the downsampled features from frame 1 are the same size as the native scale features from frame 2. This explains the major inflection point at 5 levels.

Notice that for all three datasets, there are significant diminishing returns beyond 9 levels, and so we have decided to limit our implementation to nine levels, to keep the resource requirements reasonable.

Figure 4 examines the effects of increasing the number of pyramid levels per octave while holding total number of levels constant. KITTI 08 shows small improvements with increasing levels per octave, largely due to the increased resolution of the higher levels—there are more pixels where corners can be found.

Graffiti and Boat show much more significant increases, because the increase in levels means that there are more differently-scaled views of each feature, and so the probability of finding two that are very similar in scale is higher.

Since there is a downturn in the relative benefit of adding levels per octave beyond 3, we have chosen 3 levels per octave, with a scaling ratio of $2^{-\frac{1}{3}}$ between successive levels.

II. Architecture

A. Overview

Figure 5 shows a high-level overview of the hardware system as implemented (only showing four levels for clarity). A DMA controller on the FPGA fetches pixels from the main system memory across the PCI-Express bus. The scaler module (see detail in figure 8) takes these input pixels and generates filtered versions at lower resolutions.

These different scale levels each flow separately from now on. The image pixels go into a corner detection module, which uses the Harris-Stephens corner detector to determine whether a given pixel position is a corner. The process requires information from the environment of the pixel, so the output is delayed by 4 full rows of image data and 24 clocks from when that coordinate entered the module.

The dispatcher module selects an available ORB module to generate the descriptor for a given corner. Each ORB module exports a signal indicating whether its contents are valid and ready to accept input, and the dispatcher selects the first

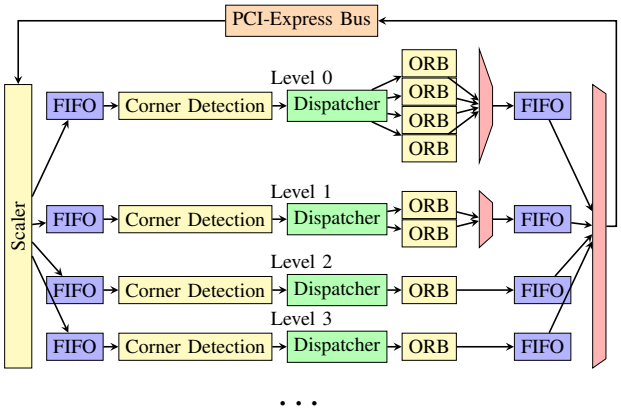
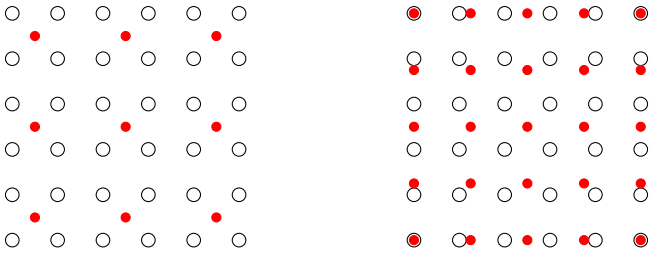


Fig. 5: An overview of the Tarsier system



(a) 1/2 filter (b) 4/5 filter

Fig. 6: Sample point arrangements for the two downsampling filters. Input: black circles, Output: red dots

module that is ready to process the corner. If all the modules at that level are unavailable, the corner is dropped.

Since a maximum of one ORB module is started per clock cycle, and the descriptor generation is a constant time operation, it is guaranteed that two descriptors from the same level cannot collide. It is possible for two descriptors from different levels to collide, so we have short FIFOs on the output ends to ensure that we don't lose any features at this point.

Finally, the descriptors are serialised for transport back to the system memory via PCI-Express by another DMA controller.

B. Downsampling

In order to generate downsampled versions of the input image, we employ a cascade of small bilinear scaling filters. Bilinear filters take four input points, arranged at the vertices of a rectangle, and output one sample, linearly interpolated in the x and y axes. A given input point will typically contribute to multiple output points.

Since we are designing downsampling filters, the output rate should be lower than the input rate. This is accomplished by having the output samples (red dots) more widely spaced than the input samples (circles), as can be seen in figure 6. The filters are 2-dimensional, so the effective average output rate is the product of the linear rates in each direction.

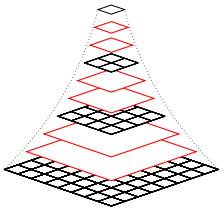


Fig. 7: Exponential image pyramid. Octave levels are in black, intermediates are in red

The image pyramid used is exponential with height, comprising three octaves, where each octave has linear dimensions half that of the previous octave (e.g. if octave 0 has dimensions 640×480, octave 1 has dimensions 320×240 and octave 2 has dimensions 160×120), as shown in figure 7. Additionally, to make the scale jumps less dramatic, two intermediate levels are inserted between each octave level, evenly spaced. Since we are using three levels per octave, the ratio between the linear dimensions of two successive levels is $2^{-1/3} \approx 4/5$.

To minimise filter latency, and to avoid quality loss due to repeated filtering, the base level of each octave is calculated directly from the previous octave using a $\frac{1}{2}$ bilinear filter, with output rate $\frac{1}{4}$. The two sublevels of each octave are generated using $\frac{4}{5}$ filters, with rate $\frac{16}{25}$. This arrangement is shown in figure 8.

There are advantages to making the ratio terms of the filter small integers, as opposed to choosing more accurate ratios, since the denominator of the fraction determines the periodicity of the repetition of the filter terms, and the numerator the number of different sets of output coefficients. For instance, in the case of the ratio $\frac{1}{2}$ filter, there is only one filter kernel, which repeats every 2 input pixels in the x and y axes:

$$\begin{bmatrix} \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} \end{bmatrix}$$

This has the effect of taking squares of four pixels, averaging them and outputting the result.

The ratio $\frac{4}{5}$ filter, on the other hand, has 16 different sets of 4 filter coefficients, which repeat every 5 input pixels in the x and y axes.

The filters themselves are implemented using a 2×2 sliding window and precomputed filter coefficients, which are determined based on the position inside the repeating block. Both the input and output for the module constitute a single pixel, its coordinates and a validity signal. All images are streamed through the filters, allowing for minimal latency. An output pixel will be available as soon as all the input information is available (excluding pipeline delays of a handful of clock cycles), and all the levels of the pyramid can be processed in parallel.

C. ORB Modules

The Tarsier system uses discrete ORB processing modules (see figure 9) to generate ORB descriptors. Each module

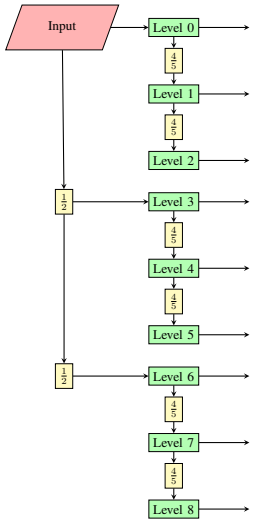


Fig. 8: Data flow in the scaling system

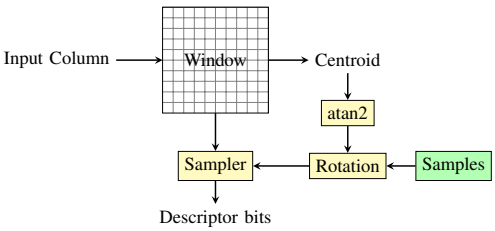


Fig. 9: The ORB Module

contains a 37×37 sliding window shift register, into which columns of input pixels are shifted. This window holds the local environment of the corner, and is sampled to generate bits of the descriptor. The input to the Harris-Stephens module is delayed slightly so that the signal indicating a detected corner perfectly aligns with that pixel coordinate being in the centre of the sliding window.

Each module requires 43 consecutive columns of input to fill the sliding window, so that it is ready to calculate a descriptor (37 columns of input data + 1 37-element pipelined adder tree). Input continues to be shifted into the window while it waits for a trigger. The shift register is implemented using a RAM, so that we can perform random access as well, for sampling pixels.

When triggered by the dispatcher, the ORB module stops accepting input and starts calculating descriptor bits. The descriptor generation takes 266 clock cycles (6 cycles to calculate an angle estimate + 3 cycles to rotate coordinates + 256 comparisons + 1 cycle memory latency). While generating descriptor bits, the module is unable to accept input, and so must be refilled with image data before it can process further corners.

The ORB module keeps a live calculation of the image centroid position, updating it with every incoming and outgoing column of pixels. Calculation of the arctan of the centroid is done with a 64-element lookup table, which takes 6 clock

TABLE I: Allocation of ORB processing modules to pyramid levels in the hardware design

Level	Linear Scale	Pixel Rate	Modules
0	1.000	1.000	4
1	0.800	0.640	3
2	0.640	0.410	2
3	0.500	0.250	2
4	0.400	0.160	1
5	0.320	0.102	1
6	0.250	0.063	1
7	0.200	0.040	1
8	0.160	0.026	1

cycles. The sines and cosines of these quantised angle values are stored in another LUT. Instead of rotating the image and taking samples at fixed coordinates, we do the reverse, leaving the image fixed and rotating the sample coordinates. This is performed by a single vector rotation unit for each of the 256 sample pairs in sequence, which take 3 clock cycles to calculate:

$$\begin{bmatrix} rx \\ ry \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

These rotated sample points are used as coordinates into the sliding window RAM to access the sample pixels.

D. ORB Module Duplication

If we just had one processing module per pyramid level, we would find ourselves dropping any corners that arrive while the module is processing or refilling, potentially as many as 154 for every corner accepted (they can occur every second pixel). Since no input is required to sustain feature extraction, we place a FIFO buffer ahead of the corner detector, so that input can be stalled while the processor is running. While this buffering prevents missed features while the module is processing, it cannot stop features being missed while the module is refilling.

We also duplicate the ORB modules in the lower levels of the pyramid, where the pixel rates are high. Each level of the pyramid will have multiple instances of the ORB module, and corners are allocated to available modules as they are detected. As long as at least one module is ready when a corner is detected, the corner is not dropped. If, however, every module at a given level is in use, either filling or generating, the corner must be dropped.

We allocate the limited processor modules to pyramid levels in approximate proportion to their pixel rates, as shown in table I.

E. Buffering

There are several possible buffering strategies. If no buffering is implemented at all, any corners arriving while no modules are available will simply be dropped. The consequences of this are not catastrophic, as typically only one pixel in 200 is a corner, and moreover, ORB-SLAM does not require every single corner to be matched successfully to estimate position accurately. This approach has the advantage of hard realtime performance and minimal latency.

Alternatively, buffering can be implemented such that the input is stalled only when all the ORB modules at a level are generating descriptors. If any modules are filling or ready, the input is allowed to flow. This is the approach that we have selected for implementation in hardware. Any module that is not running is accepting input, and once it has received 43 clock cycles of input, it becomes able to generate the descriptor for a corner once more. If all the modules are generating, we stall the input. If a module has finished processing and needs to refill to be able to run again, though, we must resume input to provide pixels to the module, otherwise it will never refill. If any corners are detected while all the modules are filling or generating, they must be dropped, however, because we do not have enough information about their environment in any of the modules to generate a descriptor.

The third alternative is much more conservative, where input is stalled as soon as any single module is running. Input is resumed when all modules are filling or ready. This strategy will only drop corners if they are detected while all modules are filling, but the input will stall every single time a corner is detected, rapidly overflowing all but the largest input buffers, and negating the high throughput design.

It is important to recognise that higher levels of the pyramid have lower pixel rates than lower levels, and thus input arrives more slowly. This means that while descriptor generation at level 0 might take 266 input pixels to complete, at level 3 only 66 input pixels will arrive on average, and at level 6 only 16. The upshot of this is that from level 4 onwards, filling the module takes longer than calculating the descriptor.

This corner dropping is a design decision that is necessary for the extremely high throughput of the device. To some extent, it can be mitigated by having more modules at each level. In future we would like to investigate the effects of different choices of buffer length and strategy on the drop rate and matching performance on a real-world dataset.

F. ORB Module Sharing

It would be advantageous if we could simply have a pool of ORB modules shared between all of the levels, and whenever a corner is detected on any level, one free module is selected to process that corner. This would lead to more effective resource utilisation, as a corner could not be dropped unless every single module was unavailable.

The most effective means of doing this would be to store the environment of a corner and replay it into a selected module when the corner is detected. The increase in latency would be modest (no more than 50 clock cycles) and it would enable more efficient utilisation. However, the routing between the scaling modules and the ORB modules would be expansive. The input to the ORB module is column-by-column, so the input bus is $37 \times 8 = 296$ bits wide. Moreover, since the ORB modules need 43 consecutive columns of input, the routing

TABLE II: Hardware Resource and Performance Summary. Results are for 3 octaves with 3 levels per octave, 16 ORB modules total, allocated as in table I.

Logic Elements	206,000
Registers	231,973
Arria V ALMs	114,049 (83%)
DSP Elements	449 (43%)
Memory Bits	8,579,952 (49%)
f_{max} (slow model)	150 MHz
f_{max} (fast model)	230 MHz
TDP	5340 mW

TABLE III: Performance Comparison

Device	Throughput	Frame Latency	1080p FPS	Energy per frame (approx)
CPU (1 thread)	28 MPix/s	74.1 ms	13.5	405 mJ
CPU (4 threads)	100 MPix/s	74.1 ms	48.2	430 mJ
GPU	40 MPix/s	52.1 ms	19.2	15.5 J
Tarsier (slow model)	150 MPix/s	13.8 ms	72.3	74 mJ
Tarsier (fast model)	230 MPix/s	9.1 ms	110.9	57 mJ

would have to be a $296 \times 9 \times 16$ crossbar switch, which is well beyond the capabilities of the Arria V FPGA we are using.

III. Results

A. Resource Usage

We are currently using as much of the Arria V FPGA as is practical, managing to fit 16 processing modules spread across 9 levels. As shown in table II, we are using 83% of the ALMs available—any more and we found the routing delays became excessive and performance elsewhere suffered, as well as doubling compilation time. Availability of the other resources—memory, DSP blocks, etc—does not constrain the design at all.

The "slow model" represents performance on a worst-case silicon FPGA, whereas the "fast model" is the performance possible on good quality silicon, as reported by TimeQuest.

B. Comparison with CPU and GPU

We devised a benchmark for the purpose of comparing the performance of the hardware implementation to the readily-available alternatives. We load 500 pairs of stereo frames into system RAM from sequence 8 of the KITTI dataset [10], then time how long each implementation takes generate the image pyramid, find the corners and generate their descriptors, including copy time where relevant (i.e. on all platforms except CPU). Note that no matching is performed on any of the platforms.

For the CPU and GPU implementations, we used OpenCV 3.2 on Windows 10 amd64, running on a quad-core i5-2500K (Sandy Bridge) clocked at 4.3 GHz, with a GTX 970 running the CUDA kernels. Both OpenCV algorithms use FAST as an early cull on the corners, and only after they have been filtered do they determine corner scores with the Harris-Stephens detector. As explained in section I-D2, the hardware implementation does not use FAST. It is possible that FAST will exclude corners that Harris-Stephens will not, however, but this should result in a faster speed for OpenCV.

Frames in KITTI sequence 8 are of dimension 1226×370 and 8-bit greyscale. The Tarsier library and driver use asynchronous (non-blocking) processing, where multiple images are queued and processed while the CPU is idle. The CPU implementation is synchronous (blocking), because there is no other work to be done at the same time, and the CUDA implementation is also blocking, because the OpenCV asynchronous implementation did not run successfully.

Table III shows a comparison of the performances of the various implementations. Frame latency is here defined as the time taken between submitting a frame of size 1920×1080 for processing and when the descriptors are returned. Energy per frame is calculated using the available TDP figures for the various devices, dividing them by the throughput and multiplying by the size of an image.

As can be seen, the Tarsier offers outstanding latency (< 16 ms allows realtime 60 fps video processing) and throughput, while using very low energy per frame.

IV. Conclusion

At present, the Tarsier device performs excellently—it successfully generates descriptors for real images with sufficiently low latency for integration into a 1080p resolution 60 fps loop.

Thanks to the pipelined design (corners are being detected and descriptors calculated while the input data is being transferred to the Tarsier by DMA), the system latency is within 1% of the theoretical minimum for the core clock speed, and the throughput is extremely close to 1 pixel/cycle. With a core clock of 150 MHz, we see a throughput of 149 MPixel/s. Since most commercial cameras of reasonable cost are able to sustain no more than 1080p60, or 124 MPixel/s, we consider this to be an adequate level of performance.

A. Scaling

There are three main ways in which this design could scale with more hardware.

- More levels — More levels allow the system to detect features at larger scales. Figure 3 shows diminishing returns beyond approximately 9 levels, but for particularly large input images (4K etc), more scale levels are likely to be more useful. Conversely, for small input images, fewer levels are necessary.
- More modules per level — Adding more ORB modules at each level of the pyramid means that fewer corners must be dropped due to resource starvation. We aim to characterise the benefit of additional modules in future work.

TABLE IV: Resource Usage by Module

Module	LCs	Memory Bits	DSPs
Per-level overhead	1350	36860	0
Harris-Stephens Corner Detection	13580	55300	5
ORB Module	4930	19000	23

- More memories per module — It is possible to duplicate the sliding window inside each ORB module so that multiple descriptor bits can be calculated each clock cycle. Since almost as much logic is required as adding a new module, and modules are more useful, we predict that increasing modules per level is a more effective use of resources, although we will investigate this in future work.

B. Experimental Setup and Performance Limitations

The host PC running the Tarsier board is furnished with dual-channel DDR4 running at a clock of 1500 MHz, so the system memory is capable of feeding data at approximately 48 GB/s. The PCI-Express link connecting the CPU to the FPGA is a Gen 2.0 x4 link, which can transfer at a maximum of 2 GB/s full duplex. The Avalon bus connecting the PCI-Express controller on the FPGA to the feature extractor core operates at a clock of 125 MHz and a data width of 32 bits, transferring at a maximum of 500 MB/s. Finally, the feature extractor core operates at a clock of 150 MHz (slow model) or 230 MHz (fast model) and a data width of 8 bits, giving a maximum core throughput of 150 MB/s or 230 MB/s respectively. Since we are using 8-bit pixels, this corresponds to 150 MPixel/s or 230 MPixel/s.

The output side of the system has slightly lower data requirements than the input side. Each descriptor requires a transfer of 40 bytes (the size of the packet), which is loaded directly into the (half-duplex) Avalon bus for transfer back to the host system over PCI-Express. Each level of the pyramid can generate feature outputs independently of each of the others. As a simplification, we assume that the probability of any given pixel at a particular level being a corner is uniform and given by p. Thus the average features generated per input pixel, q, is given by

$$q = p \times \sum_{n=0}^{8} 4^{\frac{-n}{3}}$$
$$q \approx p \times 2.66$$

Taking a representative value for p of $\frac{1}{200}$, we find that we are transferring approximately 0.53 bytes per input pixel. The total data rate over the Avalon bus is thus 230 MB/s for the slow model and 352 MB/s for the fast model.

The limiting factor on the input side is the core throughput. The most effective way to increase this would be by increasing the core clock speed—reducing the critical logic path allows the core to run glitch-free at a higher clock speed. So long as the core clock speed remains below approximately 325 MHz, it will remain the limiting factor; above this, the input and output data requirements for the core saturate the Avalon bus, making it the limiting factor. Increasing its maximum data rate is, however, as simple as widening it to 64 bits.

V. Future Work

In the near future, we would like to perform a more detailed study of the effects of adjusting the buffer size and strategy for avoiding dropped features, including an evaluation of the effects of dropping features, and how detrimental it is to the performance of ORB-SLAM.

We plan to implement matching of features from multiple images entirely within the hardware, as this is a task with significant acceleration potential, in particular performing 256-bit Hamming distance with a throughput of one per clock cycle.

Ultimately, we intend to integrate the Tarsier device into the processing loop of ORB-SLAM and investigate the acceleration that can be achieved. We would like to investigate whether logic elements are better spent on more pyramid levels or on more modules in each level in such a real-world test.

VI. Acknowledgements

This work was supported by the Australian Research Council Centre of Excellence for Robotic Vision (project number CE140100016), and through an Australian Government Research Training Program Scholarship.

References

[1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: a versatile and accurate monocular SLAM system," *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.

[2] E. Rosten and T. Drummond, "Fusing points and lines for high performance tracking," in *Computer Vision, 2005. ICCV 2005. Tenth IEEE International Conference on*, vol. 2. IEEE, 2005, pp. 1508–1515.

[3] C. Harris and M. Stephens, "A combined corner and edge detector." in *Alvey vision conference*, vol. 15. Manchester, UK, 1988, p. 50.

[4] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "Orb: an efficient alternative to sift or surf," in *Computer Vision (ICCV), 2011 IEEE International Conference on*. IEEE, 2011, pp. 2564–2571.

[5] K.-Y. Lee, "A design of an optimized orb accelerator for real-time feature detection," *International Journal of Control & Automation*, vol. 7, no. 3, 2014.

[6] J. Weberruss, L. Kleeman, and T. Drummond, "Orb feature extraction and matching in hardware," in *Australiasian Conference on Robotics and Automation, proceedings of the*. Australian Robotics and Automation Association, 2015.

[7] D. Bouris, A. Nikitakis, and I. Papaefstathiou, "Fast and efficient fpga-based feature detection employing the surf algorithm," in *Field-Programmable Custom Computing Machines (FCCM)*, 2010.

[8] O. Ulusel, C. Picardo, C. B. Harris, S. Reda, and R. I. Bahar, "Hardware acceleration of feature detection and description algorithms on low-power embedded platforms," in *Field Programmable Logic and Applications (FPL), 2016 26th International Conference on*, 2016.

[9] S. Tanimoto and T. Pavlidis, "A hierarchical data structure for picture processing," *Computer graphics and image processing*, vol. 4, no. 2, pp. 104–119, 1975.

[10] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The kitti dataset," *International Journal of Robotics Research (IJRR)*, 2013.

[11] Oxford Visual Geometry Group. (2004) Affine covariant regions datasets. [Online]. Available: http://www.robots.ox.ac.uk/~vgg/data/data-aff.html